



Linux* Interrupt Handling

Intel® China Software Center

SSG Core Software Division



Agenda

- **Interrupt and Exception Overview**
- **Programmable Interrupt Controller**
 - 8259
 - Local APIC and I/O APIC
- **Exception Handling**
- **Interrupt Handling**
- **Softirq and Tasklet**
- **Work Queue**

Agenda

- **Interrupt and Exception Overview**
- **Programmable Interrupt Controller**
 - 8259
 - Local APIC and I/O APIC
- **Exception Handling**
- **Interrupt Handling**
- **Softirq and Tasklet**
- **Work Queue**

Interrupt and Exception Overview

- **Interrupt**

- Typically occur at random times
- Software executing INT n instruction

- **Exception**

- Error condition while executing an instruction
- Machine check exceptions

- **Interrupt and Exception Vector**

- 0 ~ 31: Reserved by IA-32 architecture
- 32 ~ 255: User-defined

Sources of Interrupt and Exception

- **Source of Interrupt**

- External I/O device through I/O APIC and Local APIC
- Software generated:
 - INT n instruction (n:0~255)
 - INT 2(NMI)

- **Source of Exception**

- Processor-detected program error
- Software generated
 - Instructions such as INTO, INT 3, BOUND, etc
 - Processor does not push an error code on the stack
- Machine check exceptions

Exception Classification

- **Faults**

- Can generally be corrected
- Allow program to be restarted
- Return address: the faulting instruction

- **Traps**

- Return address: instruction to be executed after the trapping

- **Aborts**

- Does NOT always report the precise location of the instruction causing the exception
- Does NOT allow restart of the program

Enabling and Disabling Interrupts

- **EFLAGS:IF flag**

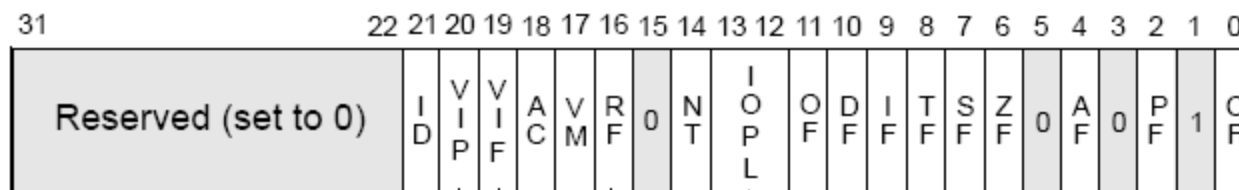
- inhibit/allow interrupt delivered to INTR or local APIC
- Does NOT affect NMI or processor generated exceptions

- **EFLAGS:RF flag**

- Control the responses to instruction breakpoint conditions (generate debug exception or not)

- **EFLAGS:TF Flag**

- Control the single step breakpoint exception



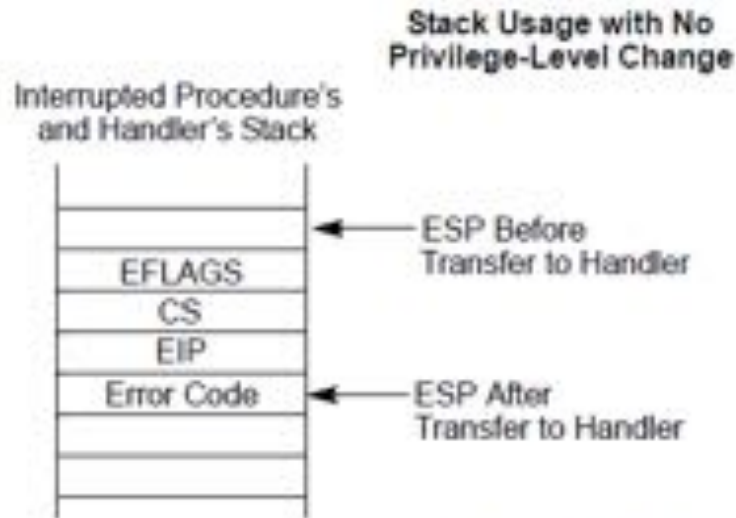
Interrupt Descriptor Table

- **Array of 8-byte descriptors**
 - Entry number is no more than 256
- **Each descriptor can be**
 - Task-gate
 - Interrupt-gate
 - Trap-gate descriptor
- **IDTR**
 - Contain the **linear address** of interrupt descriptor table

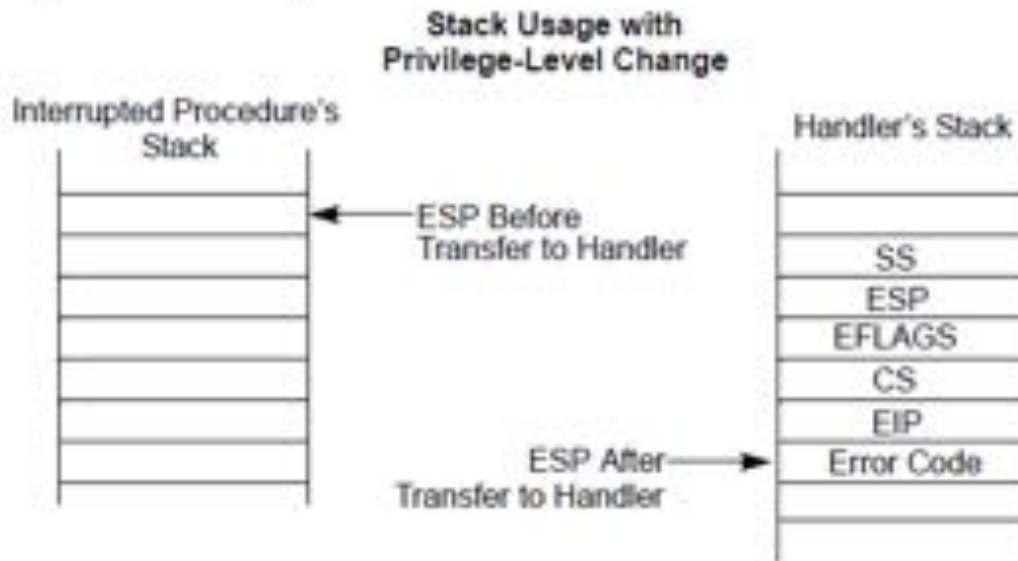
Hardware Handling of Interrupt and Exception

- **Determines the interrupt vector i**
- **Read the i entry of IDT**
- **Check Privilege level, raise GP fault if $CPL < DPL$**
- **If $CPL \neq DPL$, perform stack switch**
- **Push eflags, cs and eip in stack**
- **If exception with error code, push error to stack**
- **Load cs and eip from IDT, jump to interrupt handler**

Stack Usage



No stack switch



Stack switch:

Use TSS SSx and ESPx as new stack address

Hardware Handling of Interrupt and Exception Return

- Load the cs, eip, and eflags registers with the values saved on the stack
- If a hardware error code has been pushed before, it must be popped before executing iret
- If stack switch occurs before, load ss and esp from stack to switch stack back

Agenda

- Interrupt and Exception Overview
- **Programmable Interrupt Controller**
 - 8259A
 - Local APIC and I/O APIC
- Exception Handling
- Interrupt Handling
- Softirq and Tasklet
- Work Queue

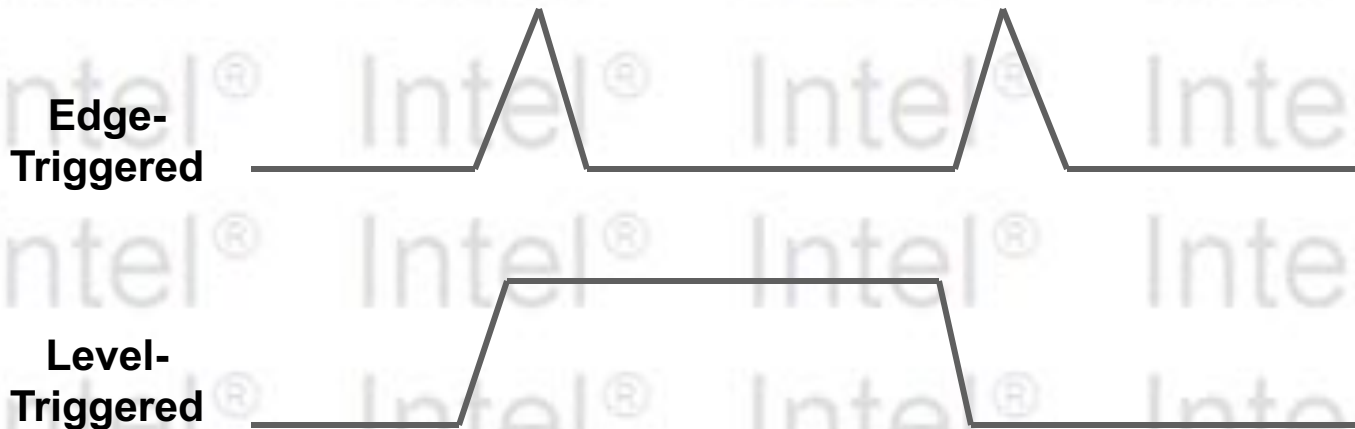
Interrupt Trigger Mode: Level/Edge

- **Edge-Triggered**

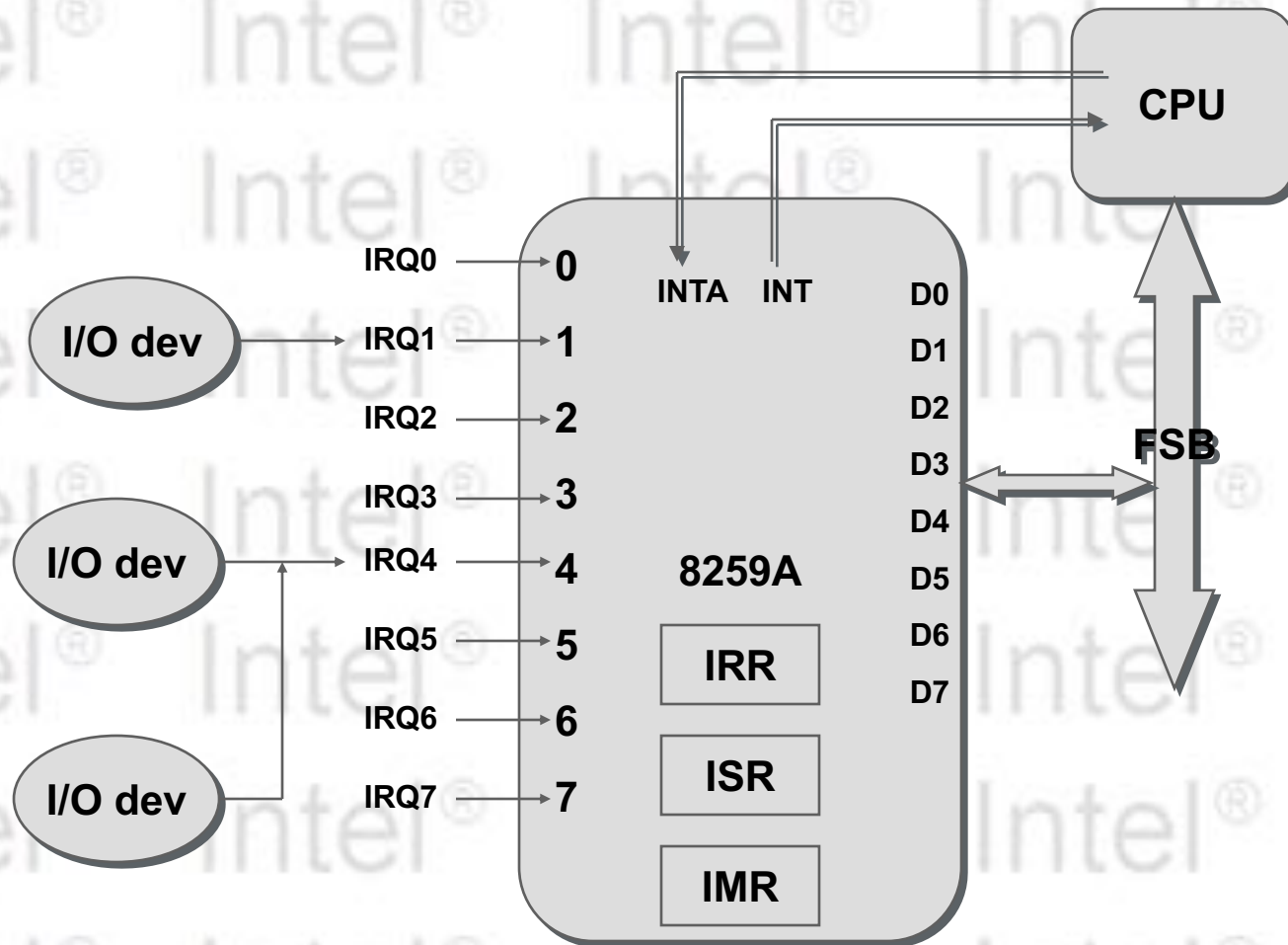
- A transient voltage change represent a interrupt
- Limitation: hard to share interrupt, prone to noisy
- Old device (ISA) use this

- **Level-Triggered**

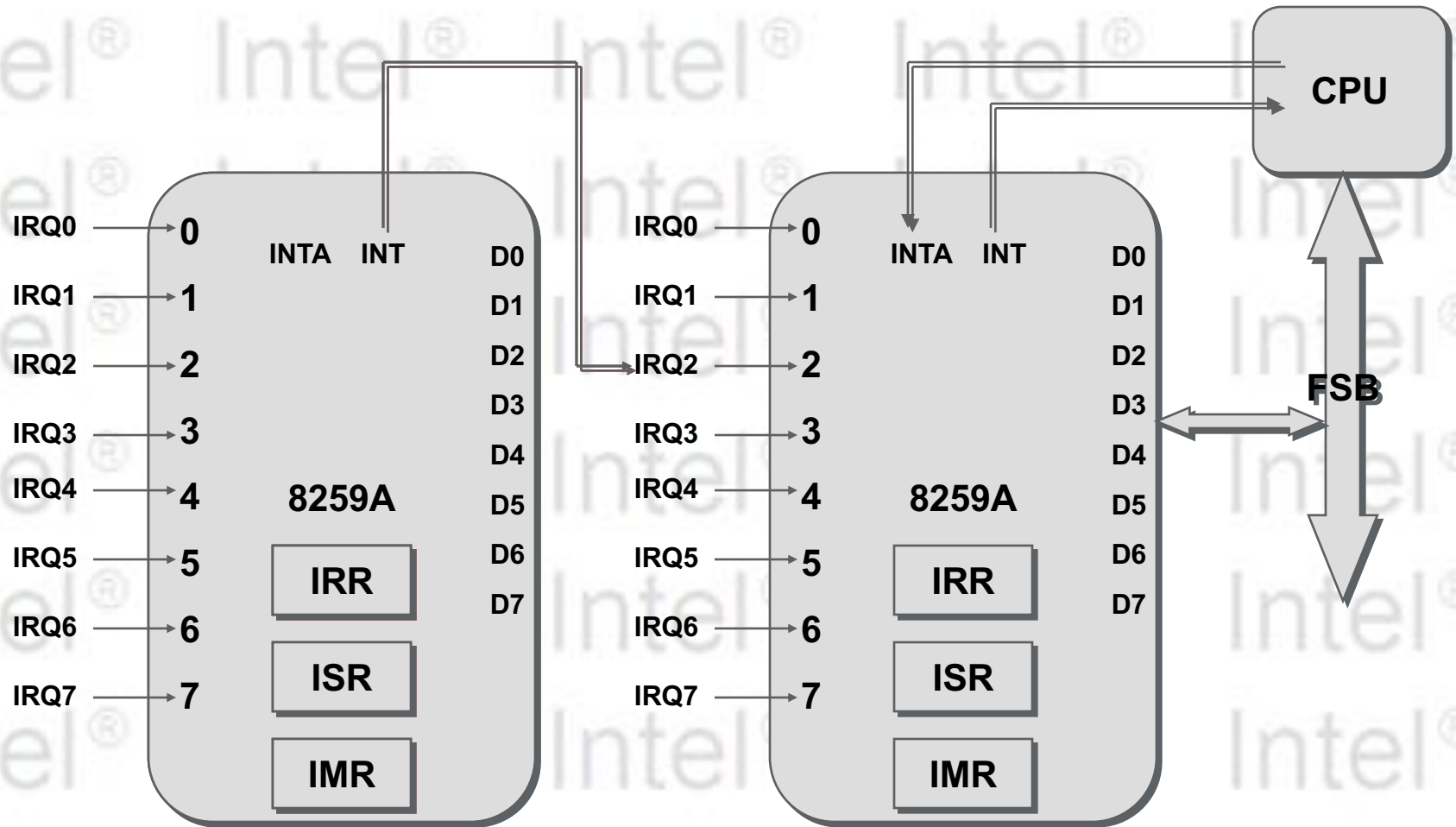
- A continuation voltage change represent a interrupt
- Easy to share interrupt
- Most modern devices (PCI) use this



8259A Overview



Two 8259As Cascade



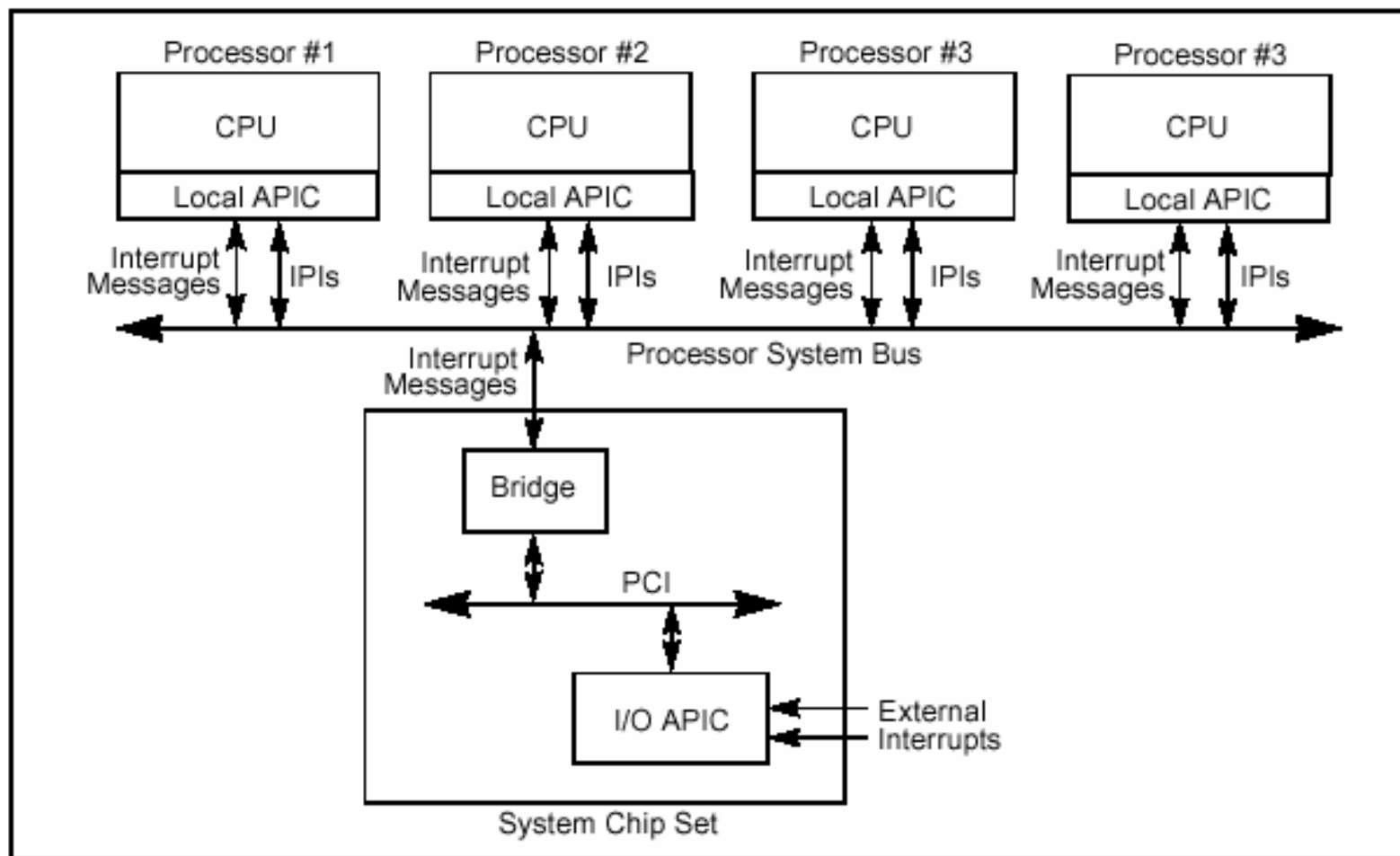
PIC Interrupt Handling Sequence

- I/O dev assert IRQ x pin
- PIC set IRR bit and send INT signal to CPU
- CPU ack the INT signal by sending INTA to PIC
- PIC then set ISR bit and clear IRR bit
- PIC initialize a second INTA to tell CPU the vector number
- CPU get vector num, start the ISR
- At the end of ISR, ISR issue EOI
- Upon receiving EOI, PIC clear the ISR bit and start to accept another interrupt.

8259A Operations

- **IRR: Interrupt request register**
- **ISR: In-Service Register**
- **IMR: Interrupt Mask Register**
- **ICW (Initialization Command Words)**
 - CPU Issue ICW1-ICW4 to initialize 8259A
- **OCW (Operation Control Words)**
 - set/clear IMR
 - Issue EOI
 - Read IRR/ISR ...

APIC Overview

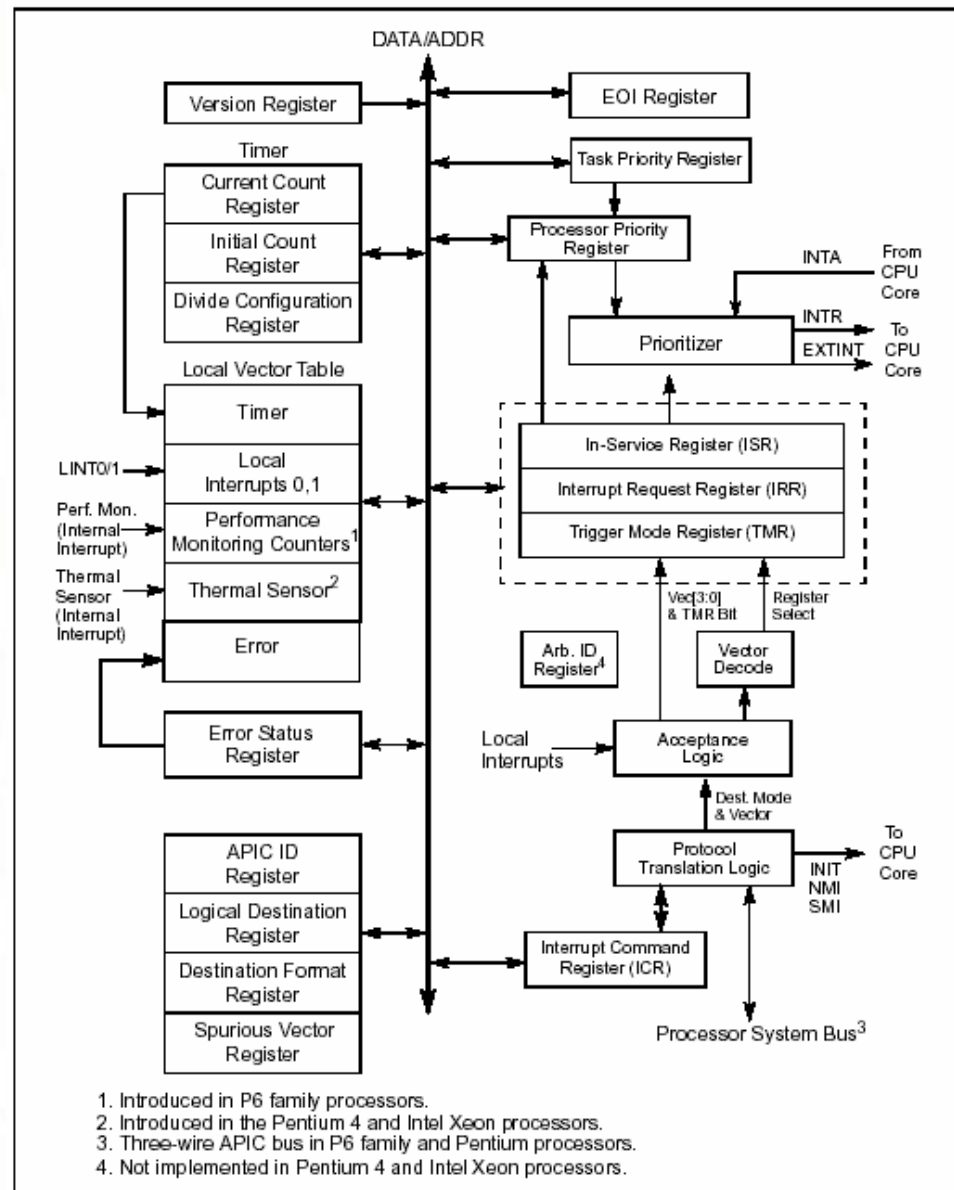


Local APICs and I/O APIC When Intel Xeon Processors Are Used in Multiple-Processor Systems

Local APIC

- **Function: receive/send interrupt**
- **Local interrupt sources**
 - Locally connected I/O devices (via LINT0/LINT1 pin)
 - APIC timer interrupt
 - Performance monitoring counter interrupt
 - Thermal Sensor interrupt
 - APIC internal error interrupt
- **External interrupt sources (IPI)**
 - Externally connected I/O devices (via I/O APIC)
 - Inter-processor interrupt (IPI)

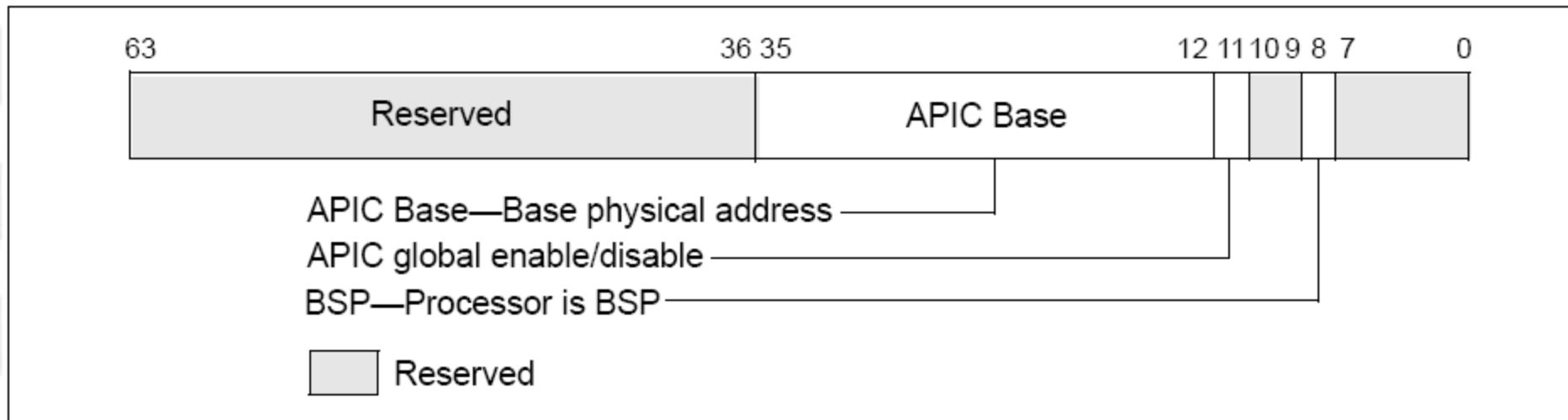
Local APIC Structure



Local APIC Structure

IA32_APIC_BASE_MSR

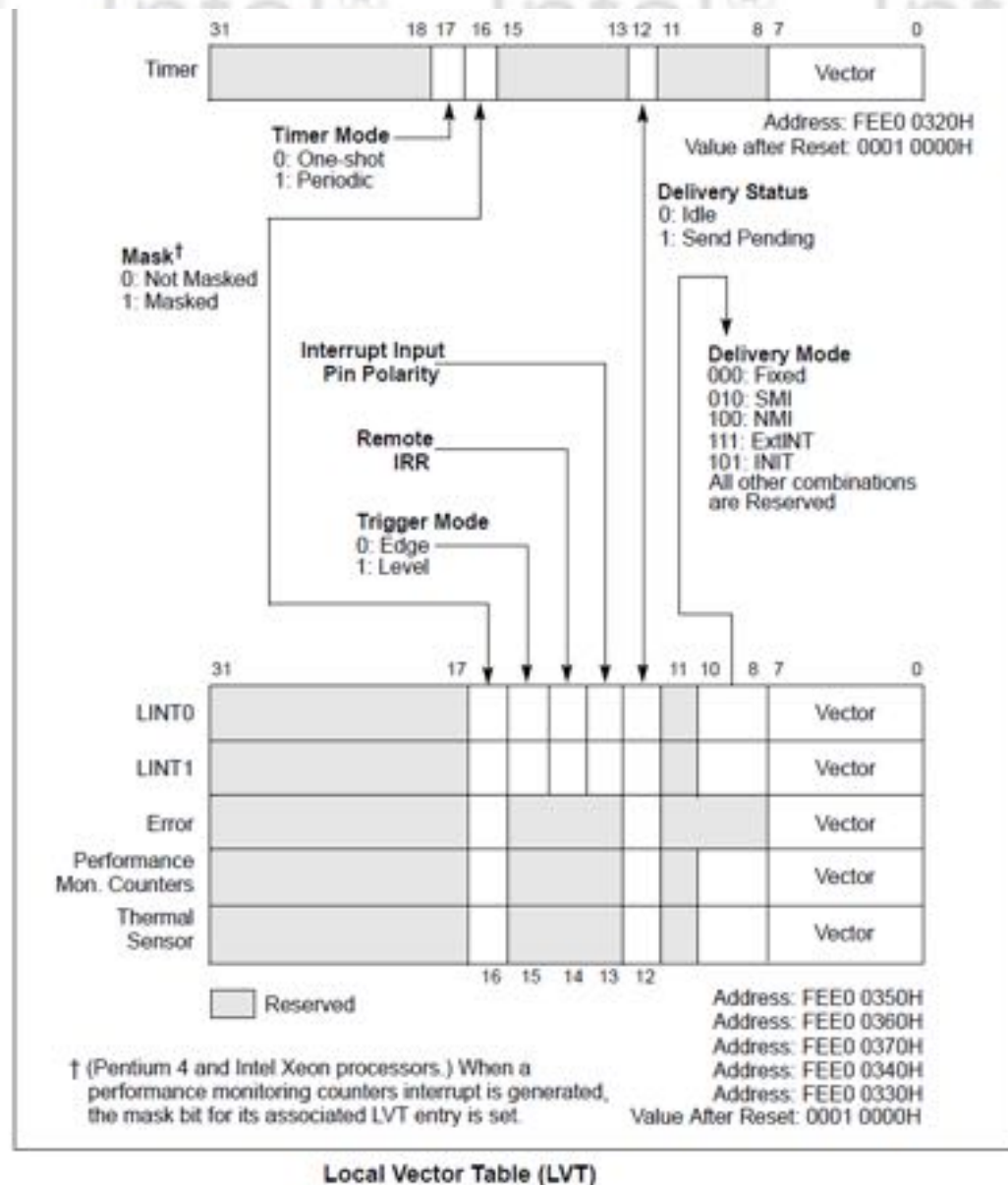
- Local APIC Register is memory-mapped register, default start at FEE0 0000H
- The start address can be changed by IA32_APIC_BASE_MSR



Local APIC ID

- **@FEE0 0020H**
- **Read only, Hardware assigned processor ID at start up, determined by hardware topology**
- **P6: 4 bits, 16 processors (thread, cores)**
- **Xeon: 8 bits, 256 processors (thread, cores)**

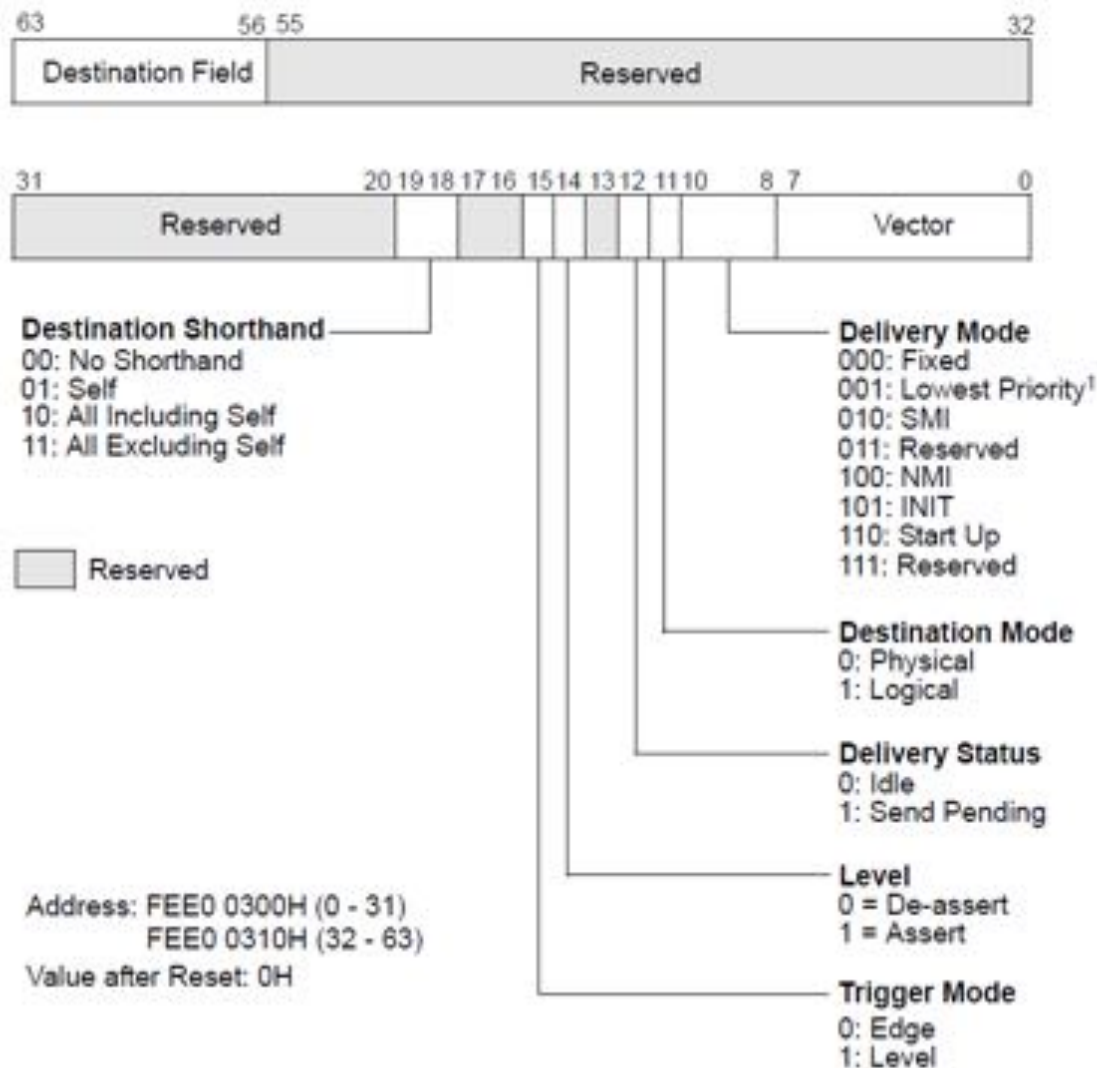
Handling Local Interrupt



Local Vector Table (LVT)

Sending IPI

- By writing to ICR



NOTE:

1. The ability of a processor to send Lowest Priority IPI is model specific.

Interrupt Command Register (ICR)

Interrupt Handling Sequence

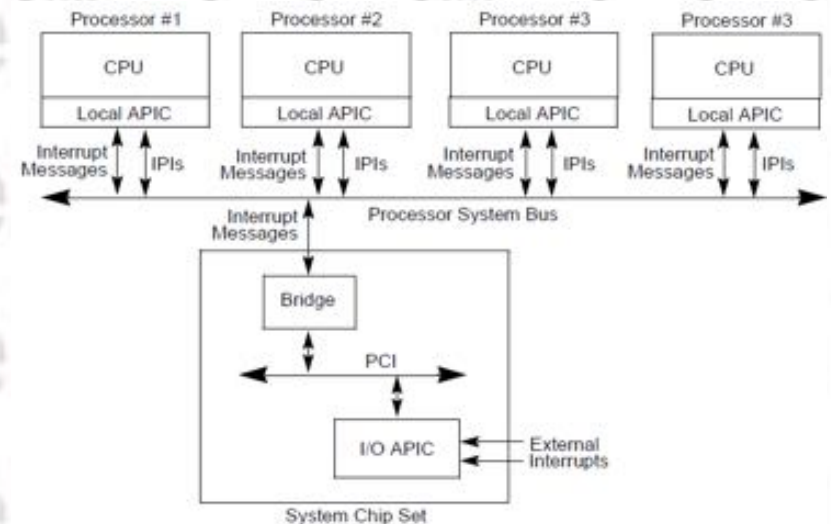
- If Interrupt is NMI, SMI, INIT, ExtINT, send directly to processor core to handling
- Otherwise, accept interrupt: set IRR/ISR to indicate pending interrupt
- Dispatch the highest pending interrupt in IRR/ISR
- When interrupt handler completes, it write EOI register. Then local APIC delete the interrupt from queue, also for level-triggered interrupt, send message to bus, indicating interrupt completion



IRR, ISR and TMR Registers

I/O APIC Interrupt sequence

- I/O device assert its pin to I/O APIC
- I/O APIC lookup the Redirection Table, use RTE info to translate interrupt pin signal into interrupt message (vector, delivery mode, trigger mode, destination processor ID)
- I/O APIC send the message to bridge controller
- Bridge controller use interrupt message to start a interrupt transaction in the FSB
- Each CPU in FSB check the processor ID, serve if match



I/O APIC Register

- **Memory Mapped Registers**
 - IOREGSEL (index)
 - IOWIN (data)
- **IOAPIC Registers**
 - IOAPICID (index: 00H)
 - IOAPICVER (index: 01H)
 - IOAPICARB (index: 02H)
 - IOREDTBL[23:0] (index: 10H-3FH)

Memory Mapped Registers For Accessing IOAPIC Registers

Memory Address	Mnemonic	Register Name	Access	D/I# Signal
FEC0 xy00h	IOREGSEL	I/O Register Select (index)	R/W	0
FEC0 xy10h	IOWIN	I/O Window (data)	R/W	1

I/O APIC Redirection Table

- **24 entry, every entry is 64 bits**
- **64 Bits definition**
 - 63:56: Destination Field (R/W)
 - 55:17: Reserved
 - 16: Interrupt Mask (R/W)
 - 15: Trigger Mode (R/W)
 - 14: Remote IRR (RO)
 - 13: Interrupt Pin Polarity (R/W)
 - 12: Delivery Status (RO)
 - 11: Destination Mode (R/W)
 - 10:8 Delivery Mode (R/W)
 - 7:0 Vector (R/W)

Agenda

- **Interrupt and Exception Overview**
- **Programmable Interrupt Controller**
 - 8259
 - Local APIC and I/O APIC
- **Exception Handling**
- **Interrupt Handling**
- **Softirq and Tasklet**
- **Work Queue**

Exception Handling sequence

- Save most registers (common)
- Call high-level C handler `do_xxx`
- Call `ret_from_exception` to exit exception

Saving Register for Exception

```
ENTRY(divide_error)
    pushl $0                # no error code
    pushl $do_divide_error  # for later use
    ALIGN
error_code:
    pushl %eds              # save common registers
    pushl %eax
    xorl %eax, %eax
    pushl %ebp
    pushl %edi
    pushl %esi
    pushl %edx
    decl %eax               # eax = -1
    pushl %ecx
    pushl %ebx
    cld                    #clear EFLAG DF
    pushl %es
    UNWIND_ESPFIX_STACK
    popl %ecx
    movl ES(%esp), %edi     # get the function address
    movl ORIG_EAX(%esp), %edx # get the error code
    movl %eax, ORIG_EAX(%esp)
    movl %ecx, ES(%esp)
    movl $(__USER_DS), %ecx
    movl %ecx, %ds
    movl %ecx, %es
    movl %esp, %eax
    movl %esp, %eax        # pt_regs pointer
    call *%edi
    jmp ret_from_exception
```

SS
ESP
EFLAGS
CS
EIP
0 (Error Code)
do_divide_error
DS
EAX
EBP
EDI
ESI
EDX
EBX

ESP →

```
struct pt_regs
{
    long ebx;
    long ecx;
    long edx;
    long esi;
    long edi;
    long ebp;
    long eax;
    int xds;
    int xes;
    long orig_eax;
    long eip;
    int xcs;
    long eflags;
    long esp;
    int xss;
};
```

Exception Handler do_xxx

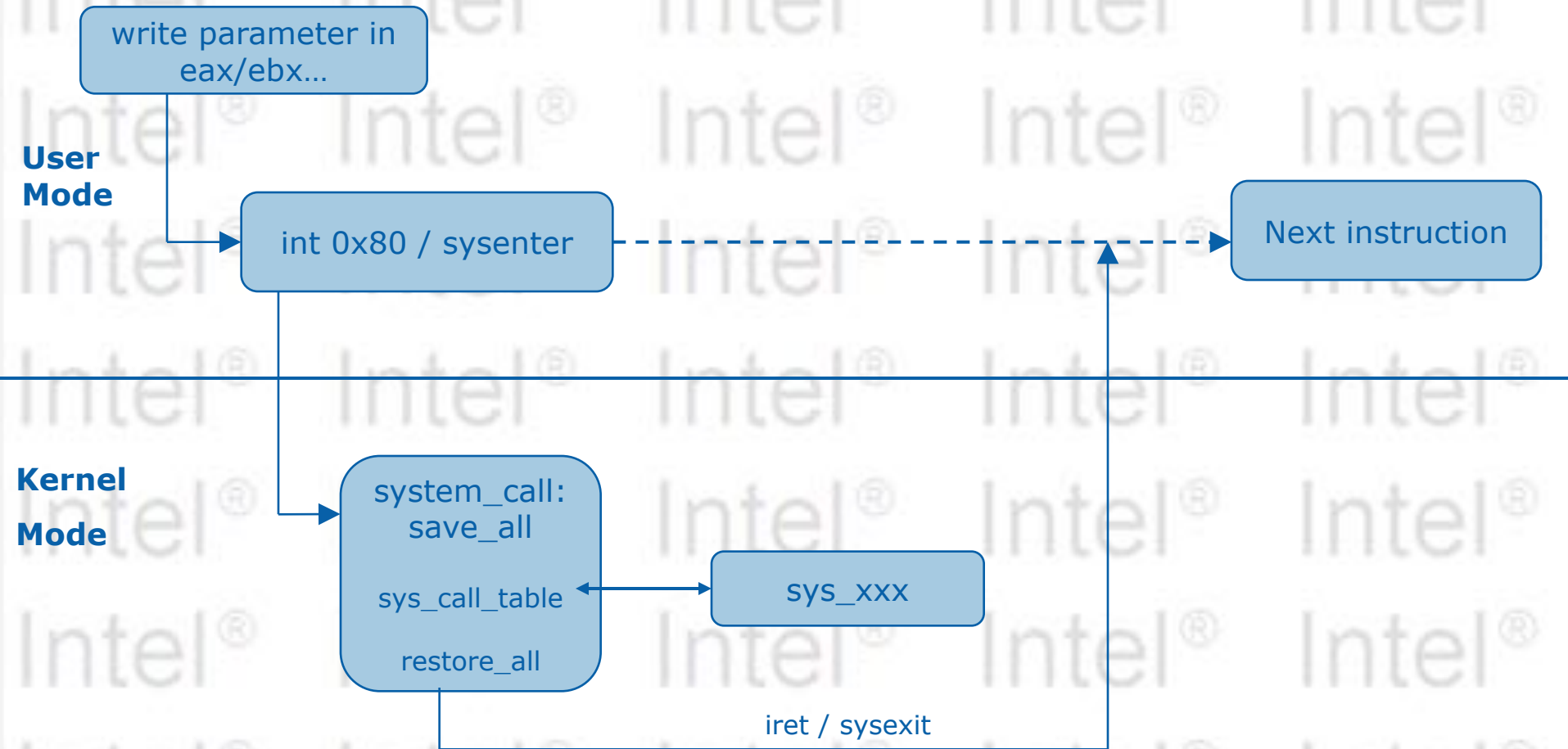
- Mostly call `do_trap` to send signal to that process:

```
current->thread.error_code = error_code;  
current->thread.trap_no = vector;  
force_sig(sig_number, current);
```

System call

- **Interfaces exported by kernel to user space**
- **Implemented as a special exception**
 - `int 0x80 / iret`
 - `sysenter/sysexit`

Systemcall Enter/Exit



Sysenter/sysexit

- **sysenter handling in CPU**

- $cs \leftarrow SYSENTER_CS_MSR$
- $eip \leftarrow SYSENTER_EIP_MSR$
- $esp \leftarrow SYSENTER_ESP_MSR$
- $ss \leftarrow SYSENTER_CS_MSR + 8$

- **sysexit handling in CPU**

- $cs \leftarrow SYSENTER_CS_MSR + 16$
- $eip \leftarrow EDX$
- $ss \leftarrow SYSENTER_CS_MSR + 24$
- $esp \leftarrow ECX$

Syscall exit

- Return value
- Error number

System call parameter passing

- **Use register instead of stack due to stack switch**
 - eax (syscall number), ebx, ecx, edx, esi, edi, ebp
 - If more than six parameter, use memory to pass
- **Pointer Parameter usage**
 - access_ok
 - get_user / put_user / copy_from_user / copy_to_user

glibc wrapper func example

- **open/write**

```
int write(int fd, const char * buf, unsigned int count)
{
    long __res;
    asm("int $0x80"
        : "=a" (__res)
        : "0" (__NR_write), "b" ((long)fd),
          "c" ((long)buf), "d" ((long)count));
    if ((unsigned long)__res >= (unsigned long)-129) {
        errno = -__res;
        __res = -1;
    }
    return (int) __res;
}
```

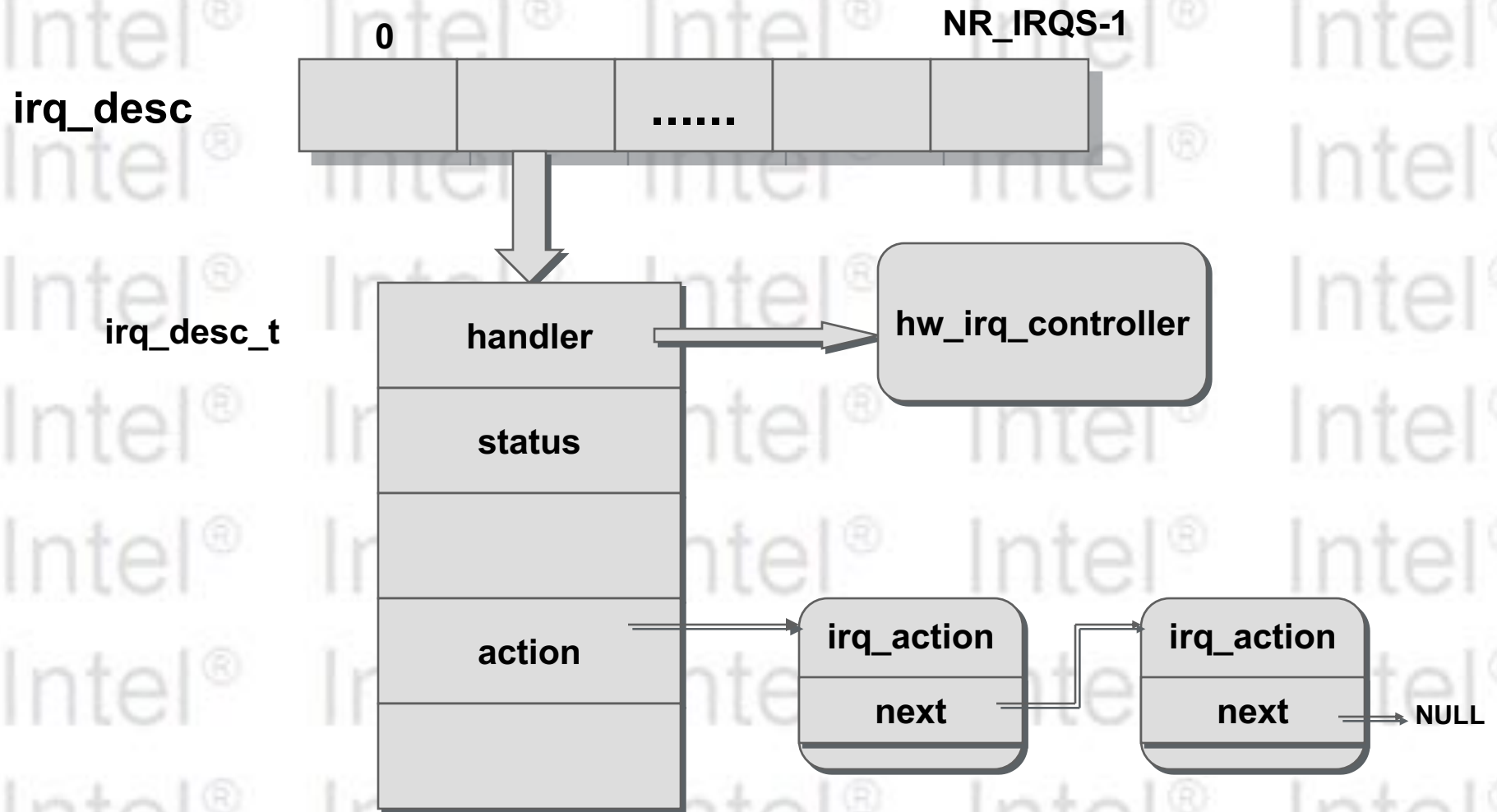
Agenda

- Interrupt and Exception Overview
- Programmable Interrupt Controller
 - 8259
 - Local APIC and I/O APIC
- Exception Handling
- **Interrupt Handling**
- Softirq and Tasklet
- Work Queue

Interrupt Handling Overview

- **Save the IRQ value and the register's contents on the Kernel Mode stack**
- **Send an acknowledgment to the PIC that is servicing the IRQ line, thus allowing it to issue further interrupts**
- **Execute the interrupt service routines (ISRs) associated with all the devices that share the IRQ.**
- **Terminate by jumping to the `ret_from_intr()` address**

IRQ Data Structure



IRQ data structure (cont)

- **irq_desc_t**

- handler: PIC object
- action:ISR
- status: state-machine (IRQ_INPROGRESS, IRQ_PENDING,...)

- **hw_interrupt_type (PIC)**

- startup/shutdown
- enable/disalbe
- ack/end

- **irqaction**

- Handler
- Flags: SA_XXXXX
- dev_id
- next

Save Context

- **Save IRQ number**

```
push $n-256 # save IRQ number
```

- **Save all the common register (except FPU, eflags, cs, eip, ss, esp)**

- **Call do_irq**

```
movl %esp, %eax  
call do_IRQ
```

```
__attribute__((regparm(3))) unsigned int do_IRQ(struct  
pt_regs *regs)
```

do_irq

- **irq_enter**
 - Inc hardirq count
- **Switch stack if stack size is 4KB**
- **_do_irq**
- **irq_exit**
 - dec hardirq count
 - do_softirq() if softirq pending and no further interrupt

_do_irq

- **ack and mask IRQ**
- **Set status to be IRQ_PENDING**
- **call handle_IRQ_event**
- **end (ack and enable IRQ)**

handle_IRQ_event

- enable interrupt (sti) (conditionally)
- Execute every irq action

```
do {  
    retval |= action->handler(irq, action->dev_id, regs);  
    action = action->next;  
} while (action);
```

- Disable interrupt again (cli) (conditionally)

IPI Handling

- **CALL_FUNCTION_VECTOR (vector 0xfb)**
- **RESCHEDULE_VECTOR (vector 0xfc)**
- **INVALIDATE_TLB_VECTOR (vector 0xfd)**
- **send_IPI_all()**
- **send_IPI_allbutself()**
- **send_IPI_self()**
- **send_IPI_mask()**

Agenda

- **Interrupt and Exception Overview**
- **Programmable Interrupt Controller**
 - 8259
 - Local APIC and I/O APIC
- **Exception Handling**
- **Interrupt Handling**
- **Softirq and Tasklet**
- **Work Queue**

Softirq Overview

- **Why Softirq**

- To serve I/O interrupt quickly

- **What is Softirq**

- 32 types of deferred function, do post-ISR work
- Running in interrupt context
- Single type softirq can run in multiple CPUs concurrently, so softirq function must be reentrant

Softirq Data struct

- **struct softirq_action softirq_vec [32]**

```
struct softirq_action  
{ void      (*action)(struct softirq_action *);  
  void      *data;  
};
```

- **Per-CPU 32 bit mask softirq_pending**

- used by local_softirq_pending()

- **preempt_count field in thread_info**

- preemption counter (bit 0~7)
- softirq counter (bit 8~15)
- hardirq counter (bit 16~27) (inc/dec by irq_enter/irq_exit)
- Used by in_interrupt()

Softirq Operation

- **open_softirq**
 - Initialize softirq function in `softirq_vec[]`
- **raise_softirq**
 - set `softirq_pending` bit, mark corresponding softirq as pending
- **do_softirq**
 - Execute the pending softirq

raise_softirq

- **Disable local interrupt (local_irq_save) : to protect data**
- **Set softirq_pending bit**
- **If possible (not in_interrupt), call wakeup_softirqd ()**
- **local_irq_restore**

do_softirq called point

- **local_bh_enable**
- **irq_exit**
 - Called by do_irq, smp_apic_timer_interrupt, smp_call_function_interrupt
- **ksoftirqd/n kernel threads**

do_softirq

- if in_interrupt() return (in ISR or softirq is disabled)
- local_irq_save to protect softirq_pending
- Switch stack if stack size is 4KB
- __do_softirq()
- restore stack if stack is switched

Tasklet Overview

- **Similarity with softirq**

- Deferred function
- Running in interrupt context

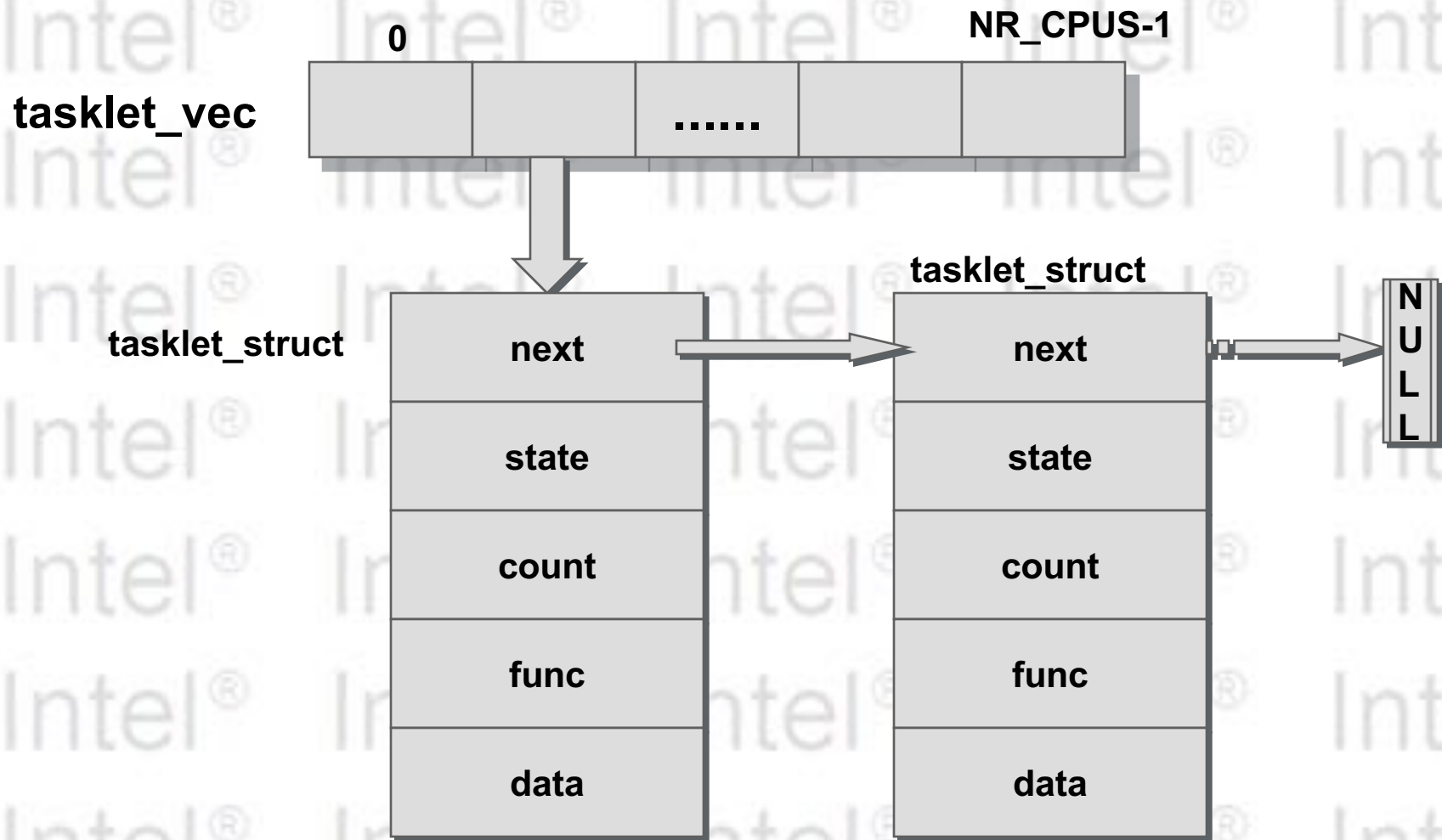
- **Difference from softirq**

- Tasklet can be created dynamically (softirq can only be created at compiling time)
- Tasklet only runs on one CPU at one time, so not to be reentrant.

- **Built upon softirq**

- Upon HI_SOFTIRQ and TASKLET_SOFTIRQ

Tasklet data structure



Tasklet function

- **DECLARE_TASKLET** (statically create)
- **Tasklet_init** (dynamicly create)
- **tasklet_schedule/tasklet_hi_schedule**
 - Schedule a tasklet for later execution
- **tasklet_disable/tasklet_enable**

Tasklet implementation

- **Schedule: tasklet_schedule**

- Check and set state TASKLET_STATE_SCHED, if already set, return
- local_irq_save: disable local interrupt to protect per-cpu data tasklet_vec
- Insert tasklet into tasklet_vec[smp_processor_id()]
- raise_softirq_irqoff: schedule the softirq
- local_irq_restore: restore local interrupt

Tasklet implementation

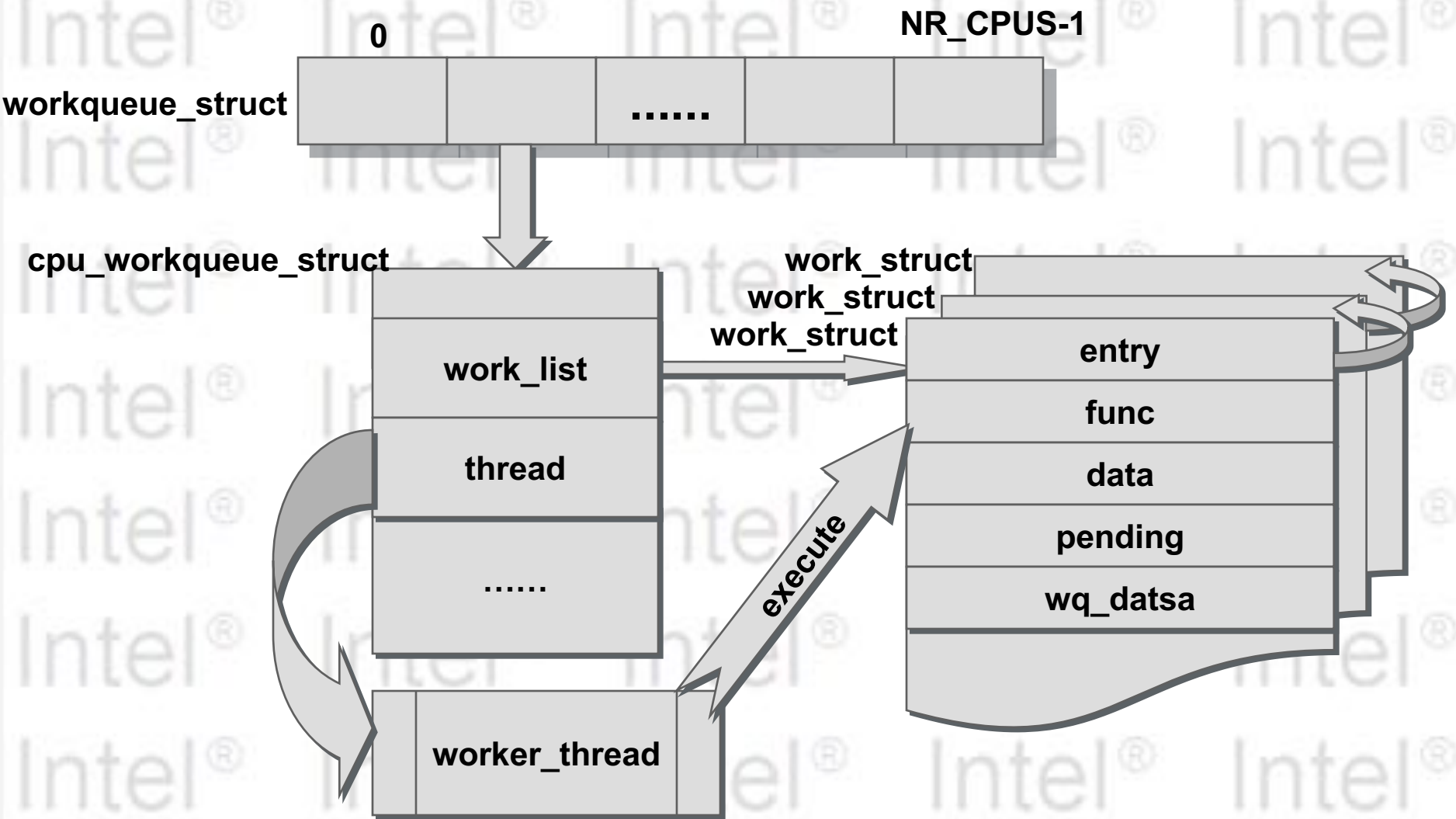
- **Execute: tasklet_action**

- Get tasklet list from tasklet_vec[n], set tasklet_vec[n] to null (disable and enable local interrupt to protect tasklet_vec)
- For every tasklet in the list:
 - If not TASKLET_STATE_RUN, and not disabled, then clear the TASKLET_STATE_SCHED status, and execute tasklet func
 - Otherwise, insert it back to tasklet_vec[n] list to execute next time

Workqueue Overview

- **Deferred function, executed by kernel thread**
- **Running in process context**
 - Can be blocked
 - Can acquire semaphore

Workqueue data structure



Workqueue function

- **create_workqueue (char* name)**
- **create_singlethread_workqueue(char* name)**
- **queue_work(struct workqueue_struct *, struct work_struct *)**
 - Add one work into work queue, using current CPU
- **queue_delayed_work**
- **flush_workqueue**

Predefined work queue

- **Kernel has predefine work queue**
 - keventd_wq: kevent/0, kevent/1, ... kevent/n
- **Function:**
 - schedule_work(w)
 - schedule_delayed_work(w,d)
 - flush_scheduled_work ()

Reference

- **Understanding the Linux Kernel, 3rd Edition,
By Daniel P. Bovet, Marco Cesati**
- **Linux Kernel Development, 2nd Edition,
By Robert Love**
- **IA32 Intel Architecture Software Developer's Manual
(Volume 3: System Programming Guide)**
- **Intel 8259A PROGRAMMABLE INTERRUPT CONTROLLER
(8259A/8259A-2)**
- **Intel 82093AA I/O Advanced Programmable Interrupt
Controller (I/O APIC) Datasheet**

Reference Code (2.6.16)

- `arch/i386/kernel/{entry.S,i8259.c,io_apic.c, irq.c,...}`
- `kernel/irq/*`
- `kernel/{softirq.c,work_queue.c}`

Q/A



BACKUP



Interrupt Vector

Protected-Mode Exceptions and Interrupts

Vector No.	Mnemonic	Description	Type	Error Code	Source
0	#DE	Divide Error	Fault	No	DIV and IDIV instructions.
1	#DB	RESERVED	Fault/Trap	No	For Intel use only.
2	—	NMI Interrupt	Interrupt	No	Nonmaskable external interrupt.
3	#BP	Breakpoint	Trap	No	INT 3 instruction.
4	#OF	Overflow	Trap	No	INTO instruction.
5	#BR	BOUND Range Exceeded	Fault	No	BOUND instruction.
6	#UD	Invalid Opcode (Undefined Opcode)	Fault	No	UD2 instruction or reserved opcode. ¹
7	#NM	Device Not Available (No Math Coprocessor)	Fault	No	Floating-point or WAIT/FWAIT instruction.
8	#DF	Double Fault	Abort	Yes (zero)	Any instruction that can generate an exception, an NMI, or an INTR.
9	—	Coprocessor Segment Overrun (reserved)	Fault	No	Floating-point instruction. ²
10	#TS	Invalid TSS	Fault	Yes	Task switch or TSS access.
11	#NP	Segment Not Present	Fault	Yes	Loading segment registers or accessing system segments.
12	#SS	Stack-Segment Fault	Fault	Yes	Stack operations and SS register loads.
13	#GP	General Protection	Fault	Yes	Any memory reference and other protection checks.
14	#PF	Page Fault	Fault	Yes	Any memory reference.
15	—	(Intel reserved. Do not use.)	—	No	—
16	#MF	x87 FPU Floating-Point Error (Math Fault)	Fault	No	x87 FPU floating-point or WAIT/FWAIT instruction.
17	#AC	Alignment Check	Fault	Yes (Zero)	Any data reference in memory. ³
18	#MC	Machine Check	Abort	No	Error codes (if any) and source are model dependent. ⁴
19	#XF	SIMD Floating-Point Exception	Fault	No	SSE/SSE2/SSE3 floating-point instructions ⁵
20-31	—	Intel reserved. Do not use.	—	—	—
32-255	—	User Defined (Non-reserved) Interrupts	Interrupt	—	External interrupt or INT n instruction.

